

به نام خدا



دانشگاه تهران



دانشکده مهندسی برق و کامپیوتر

درس امنیت سخت افزار

تمرین دوم

نام و نام خانوادگی	بهزاد جنتی
شماره دانشجویی	810103098
تاریخ ارسال گزارش	1404/02/19
استاد	دکتر سیامک محمدی
ترم	بهار 1404

5	بخش اول: Arbiter PUF
5	نصب لایبرری
6	نمونه گیری از PUF و ایجاد CRP table
8	فایل های challenge.csv و response.csv:
8	ارزیابی Reliability برای Arbiter Puf:
10	ارزیابی Uniqueness برای Arbiter Puf:
11	ارزیابی Uniformity برای Arbiter Puf:
13	ارزیابی نویز برای Arbiter Puf:
14	بررسی حمله ماشین لرنینگ به Arbiter Puf:
15	بخش دوم: XOR Arbiter PUF
15	تولید XOR Arbiter Puf:
17	تولید زوج challenge-response برای XOR Arbiter Puf:
18	ارزیابی Reliability برای XOR Arbiter Puf:
19	ارزیابی Uniqueness برای XOR Arbiter Puf:
20	ارزیابی Uniformity برای XOR Arbiter Puf:
21	ارزیابی نویز برای XOR Arbiter Puf:
22	بررسی حمله ماشین لرنینگ به XOR Arbiter Puf:
24	بخش سوم: مقایسه دو پاف با یکدیگر
24	بخش چهارم: تحلیل تصادفی بودن Response ها و کاربرد PUF به عنوان TRNG
26	نتایج تست برای Arbiter PUF جهت تولید TRNG از response های تولید شده:
27	نتایج تست برای XOR Arbiter PUF جهت تولید TRNG از response های تولید شده: ..

شکل‌ها

- شکل 1- نصب ابزارهای لازم..... 5
- شکل 2- ایجاد فایل arbiter_puf و ایجاد یک puf با 64 استیج..... 5
- شکل 3- نمونه گیری از Arbiter puf با 64 استیج..... 6
- شکل 4- تصویر Arbiter Puf..... 6
- شکل 5- کد ایجاد CRP برای arbiter puf..... 7
- شکل 6- خروجی فایل generate-crps.py..... 7
- شکل 7- تصویر خروجی های ذخیره شده challenge ها برای Arbiter Puf..... 8
- شکل 8- تصویر خروجی های Response های تولید شده برای Arbiter puf..... 8
- شکل 9- کد مربوط به ارزیابی reliability برای Arbiter Puf..... 9
- شکل 10- خروجی reliability برای Arbiter Puf..... 9
- شکل 11- کد برای ارزیابی Uniqueness پاف Arbiter..... 10
- شکل 12- خروجی ارزیابی Uniqueness برای Arbiter Puf..... 11
- شکل 13- ایجاد فایل و کد برای ارزیابی Uniformity برای Arbiter Puf..... 12
- شکل 14- مثال سایت pypuf برای بایاس Arbiter puf..... 12
- شکل 15- خروجی بایاس برای Arbiter Puf..... 12
- شکل 16- کد برای ایجاد نویز برای Arbiter Puf..... 13
- شکل 17- خروجی نویز برای Arbiter Puf..... 13
- شکل 18- کد برای ML attack به Arbiter Puf..... 14
- شکل 19- خروجی ML attack برای Arbiter Puf..... 15
- شکل 20- ایجاد XOR Arbiter puf..... 15
- شکل 21- خروجی فایل xor_arbiter_puf.py..... 16
- شکل 22- شکل XOR Arbiter PUF..... 16
- شکل 23- فایل generate-crps-xor..... 17
- شکل 24- ایجاد 10000 زوج challenge-response..... 17
- شکل 25- فایل evaluate_reliability_xor.py..... 18
- شکل 26- خروجی کد ارزیابی reliability برای XOR Arbiter PUF..... 18
- شکل 27- کد فایل evaluate_uniqueness_xor.py..... 19
- شکل 28- خروجی فایل evaluate_uniqueness برای XOR Arbiter PUF..... 19
- شکل 29- کد فایل evaluate_uniformity_xor.py..... 20
- شکل 30- خروجی بایاس برای xor_arbiter puf..... 20
- شکل 31- خروجی نویز برای XOR Arbiter PUF..... 21

22	 شکل 32- تصویری از xor arbiter puf
23	 شکل 33- خروجی مربوط به ml attack برای XOR Arbiter PUF
23	 شکل 34- کد فایل ML attack برای xor arbiter puf
25	 شکل 35- نحوه وارد کردن رشته Response ها در سایت
25	 شکل 36- سایت RANDOM BITSTREAM TESTER
26	 شکل 37- نتایج برای 7 آزمون اول برای Arbiter PUF
27	 شکل 38- نتایج برای 7 آزمون دوم برای Arbiter PUF
28	 شکل 39- وارد کردن رشته ریسپانس های XOR Arbiter PUF در سایت
28	 شکل 40- خروجی 7 تست اول برای XOR Arbiter PUF
29	 شکل 41- خروجی 7 تست دوم برای XOR Arbiter PUF

جدولها

جدول 1- مقایسه Arbiter PUF و XOR Arbiter PUF 24

بخش اول: Arbiter PUF

نصب لایبرری

همان طور که در صورت سوال خواسته شده است و در ابتدا برای انجام پروژه باید لایبرری و ابزارهای لازم رو نصب کنیم.

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS C:\Users\Asus> pip install pypuf numpy matplotlib scikit-learn
Requirement already satisfied: pypuf in c:\users\asus\anaconda3\lib\site-packages (2.2.0)
Requirement already satisfied: numpy in c:\users\asus\anaconda3\lib\site-packages (1.26.4)
Requirement already satisfied: matplotlib in c:\users\asus\anaconda3\lib\site-packages (3.9.2)
Requirement already satisfied: scikit-learn in c:\users\asus\anaconda3\lib\site-packages (1.5.1)
Requirement already satisfied: memory-profiler in c:\users\asus\anaconda3\lib\site-packages (from pypuf) (0.61.0)
Requirement already satisfied: pandas in c:\users\asus\anaconda3\lib\site-packages (from pypuf) (2.2.2)
Requirement already satisfied: contourpy>=1.0.1 in c:\users\asus\anaconda3\lib\site-packages (from matplotlib) (1.2.0)
Requirement already satisfied: cycler>=0.10 in c:\users\asus\anaconda3\lib\site-packages (from matplotlib) (0.11.0)
Requirement already satisfied: fonttools>=4.22.0 in c:\users\asus\anaconda3\lib\site-packages (from matplotlib) (4.51.0)
Requirement already satisfied: kiwisolver>=1.3.1 in c:\users\asus\anaconda3\lib\site-packages (from matplotlib) (1.4.4)
Requirement already satisfied: packaging>=20.0 in c:\users\asus\appdata\roaming\python\python312\site-packages (from matplotlib) (24.1)
Requirement already satisfied: pillow>=8 in c:\users\asus\anaconda3\lib\site-packages (from matplotlib) (10.4.0)
```

شکل 1- نصب ابزارهای لازم

در ادامه یک فایل با نام Arbiter_puf.py ایجاد کردیم و یک Arbiter puf با تعداد 64 استیج ایجاد کردیم.

```
arbitor_puf.py X
D: > Hardware security > CA&HWs > CA2 > arbitor_puf.py > ...
1 from pypuf.simulation import ArbiterPUF
2 # Instantiate an Arbiter PUF with 64 stages
3 n_stages = 64
4 puf = ArbiterPUF(n=n_stages)
5 print("Arbiter PUF instantiated with", n_stages, "stages.")
```

شکل 2- ایجاد فایل arbiter_puf و ایجاد یک puf با 64 استیج

در ادامه خروجی این فایل هم که نشان دهنده این است که پاف با موفقیت ایجاد شده است قرار داده شده است.

```

Interrupt | X Clear All | Restart | Jupyter Variables | Save ... | qiQOURSE (Python 3.8.20)

Connected to qiQOURSE (Python 3.8.20)

✓ from pypuf.simulation import ArbiterPUF ...
... Arbiter PUF instantiated with 64 stages.

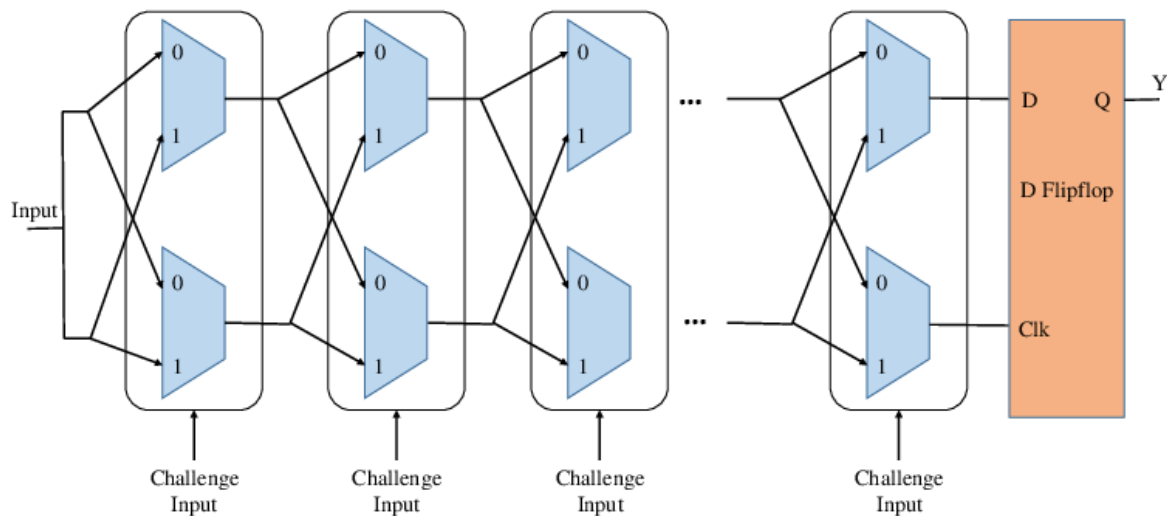
```

شکل 3- نمونه گیری از Arbiter puf با 64 استیج

نمونه گیری از PUF و ایجاد CRP table

در بخش قبل نحوه نمونه گرفتن از پاف به همراه خروجی جهت اطمینان از صحت کار رو بررسی کردیم در این بخش میخواهیم به نحوه ایجاد challenge-response pairs برای این پاف بپردازیم.

خب در ابتدا یک تصویر از Arbiter puf قرار داده شده است تا با نحوه کار آن آشنا شویم:



شکل 4- تصویر Arbiter Puf

همان طور که در درس هم بیان شده است این پاف بر اساس اینکه کدام سیگنال ورودی زودتر یک شود (Rising Edge of Inputs) کار میکند و با توجه به این مورد آگه خروجی بالا D ، flip flop ، ما زودتر از سیگنال پایینی یا کلاک برسد خروجی یک میشود و اگر دیرتر برسد خروجی صفر خواهد بود.

خب حال به سراغ ایجاد CRP ها میرویم.

طبق صورت سوال ما دو روش برای ایجاد این CRP ها داریم:

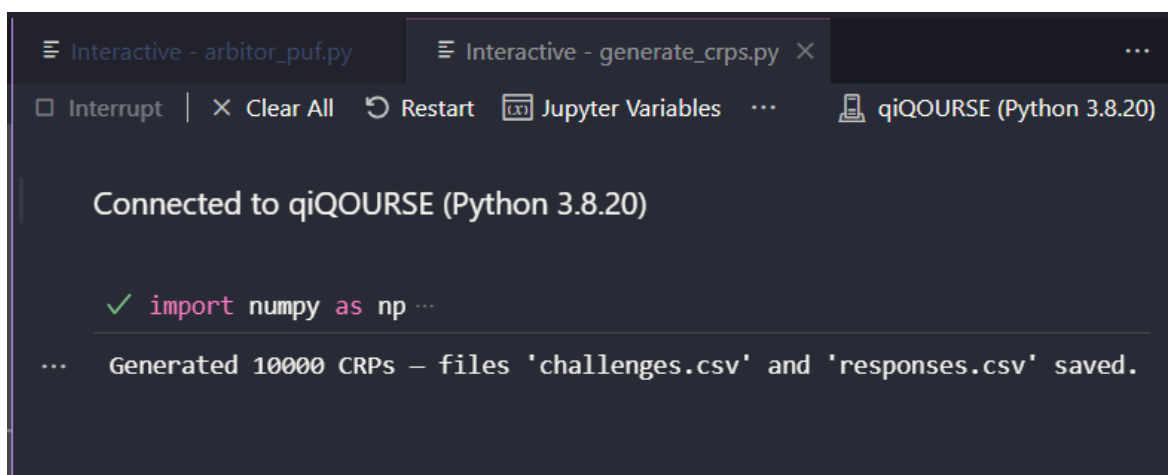
1. فایل csv شامل Challenge ها با عنوان challenges.csv به همراه یک فایل csv شامل Response های متناظر با عنوان responses.csv
 2. فایل csv شامل CRP table
- ما از روش اول استفاده کردیم.



```
1 import numpy as np
2 from pypuf.simulation import ArbiterPUF
3 n_stages = 64
4 num_crps = 10000 # Number of challenge-response pairs to generate
5 puf = ArbiterPUF(n=n_stages)
6 # Generate random binary challenges .
7 challenges = np.random.randint(0, 2, size=(num_crps, n_stages))
8 responses = puf.eval(challenges)
9 # Save challenges and responses to CSV files .
10 np.savetxt("challenges.csv", challenges, fmt="%d", delimiter=",")
11 np.savetxt("responses.csv", responses, fmt="%d", delimiter=",")
12 print(f"Generated {num_crps} CRPs – files 'challenges.csv' and 'responses.csv' saved.")
```

شکل 5- کد ایجاد CRP برای arbiter puf

همان طور که مشاهده میکنید بعد از ایجاد Arbiter Puf در این کد به شکل رندوم اقدام به ایجاد challenge با استفاده از دستور np.random کردیم و به ازای آن خروجی response رو از پاف گرفتیم. همان طور که از در صورت سوال گفته شده است ما 10000 زوج challenge-response ایجاد کردیم و خروجی هر کدام را در فایل csv مربوطه ذخیره کردیم.



```
Connected to qiQOURSE (Python 3.8.20)

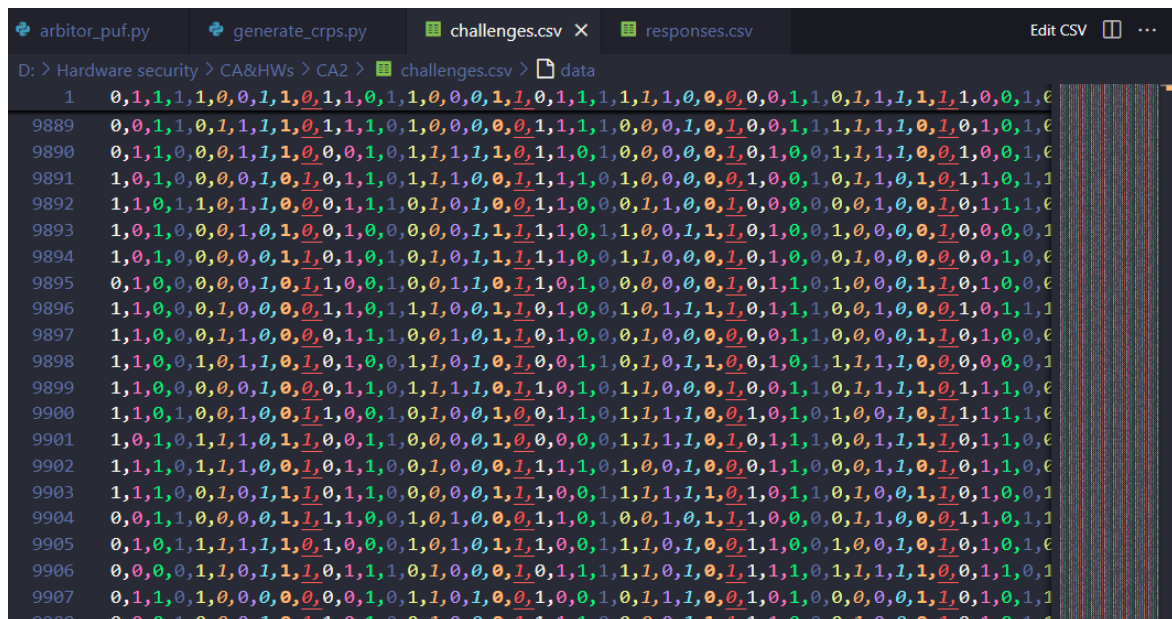
✓ import numpy as np ...

... Generated 10000 CRPs – files 'challenges.csv' and 'responses.csv' saved.
```

شکل 6- خروجی فایل generate-crps.py

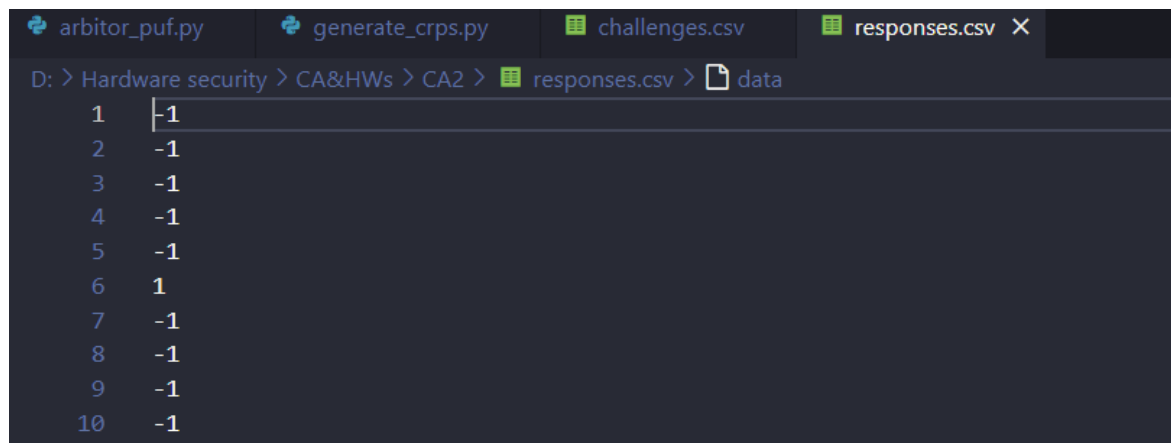
فایل‌های challenge.csv و response.csv:

در زیر می‌توانید تصویر از چلنج های ایجاد شده و ذخیره شده در فایل challenges.csv مشاهده کنید:



شکل 7- تصویر خروجی های ذخیره شده challengeها برای Arbiter Puf

همچنین تصویر از response های تولید شده برای هر چلنج که در فایل Responses.csv ذخیره شده است رو می‌توانید در زیر مشاهده کنید:



شکل 8- تصویر خروجی های Responseهای تولید شده برای Arbiter puf

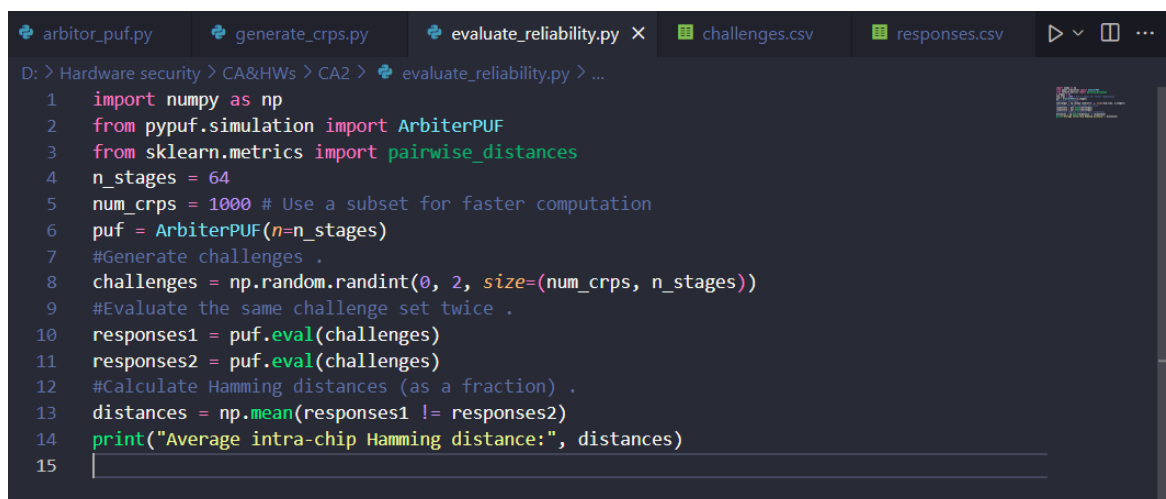
ارزیابی Reliability برای Arbiter Puf:

در بخش قرار است که Arbiter Puf خود را از نظر Reliability مورد بررسی قرار دهیم.

این معیار به تغییرات داخل یک چیپ یا همان Intra-chip میپردازد که در اینجا ما به ازای دو چلنج یکسان دوست داریم همیشه ریسپانس یکسان بگیریم (در شرایط یکسان) ولی با این حال به خاطر عوامل محیطی و خود ویژگی ها چیپ ممکن این مقدار تغییر کند.

کد مربوط به ارزیابی reliability برای Arbiter Puf:

در ابتدا یک فایل با نام evaluate_reliability.py همان طور که در صورت سوال هم گفته شده است ایجاد میکنیم.



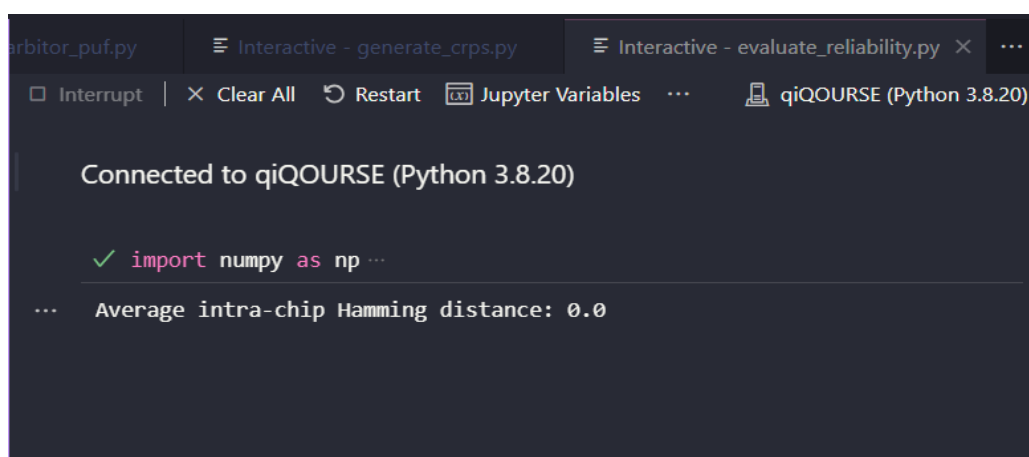
```

1 import numpy as np
2 from pypuf.simulation import ArbiterPUF
3 from sklearn.metrics import pairwise_distances
4 n_stages = 64
5 num_crps = 1000 # Use a subset for faster computation
6 puf = ArbiterPUF(n=n_stages)
7 #Generate challenges .
8 challenges = np.random.randint(0, 2, size=(num_crps, n_stages))
9 #Evaluate the same challenge set twice .
10 responses1 = puf.eval(challenges)
11 responses2 = puf.eval(challenges)
12 #Calculate Hamming distances (as a fraction) .
13 distances = np.mean(responses1 != responses2)
14 print("Average intra-chip Hamming distance:", distances)
15

```

شکل 9- کد مربوط به ارزیابی reliability برای Arbiter Puf

همان طور که در کد هم مشخص است بعد از ایجاد چلنج ها برای پاف اومدیم و دو تا ریسپانس رو بررسی کردیم که ببینیم یکسان هستند یا خیر و این کار برای ساده تر و سرعت بخشیدن به فرآیند تنها برای 1000 چلنج ایجاد شده مورد بررسی قرار گرفته است.



```

Connected to qiQOURSE (Python 3.8.20)

✓ import numpy as np ...

... Average intra-chip Hamming distance: 0.0

```

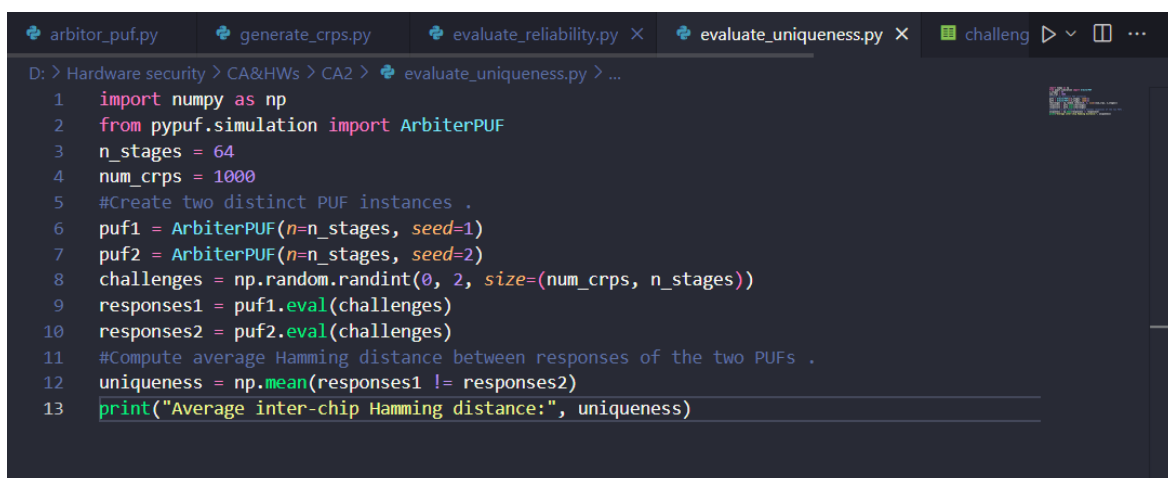
شکل 10- خروجی reliability برای Arbiter Puf

خب همان طور که در خروجی مشاهده میکنید این مقدار برابر صفر شد که دقیقا مقدار مطلوب برای ما هست چون همان طور که در درس هم گفته شد ما میخواستیم تفاوت ریسپانس ها برای چیپ های مختلف زیاد و در داخل یک چیپ کم باشد.

ارزیابی Uniqueness برای Arbiter Puf

در ابتدا به مفهوم Uniqueness در ارزیابی Puf ها میپردازیم، این مورد به این مورد به این ویژگی اشاره دارد که هر Puf باید منحصر به فرد باشد و با مابقی Puf ها فرق بکند، پس انتظار داریم که به ازای یک چلنج یکسان ریسپانس های متفاوتی از دو پاف بگیریم.

خب ابتدا یک فایل با نام evaluate_uniqueness.py ایجاد میکنیم.



```
D: > Hardware security > CA&HWs > CA2 > evaluate_uniqueness.py > ...
1 import numpy as np
2 from pypuf.simulation import ArbiterPUF
3 n_stages = 64
4 num_crps = 1000
5 #Create two distinct PUF instances .
6 puf1 = ArbiterPUF(n=n_stages, seed=1)
7 puf2 = ArbiterPUF(n=n_stages, seed=2)
8 challenges = np.random.randint(0, 2, size=(num_crps, n_stages))
9 responses1 = puf1.eval(challenges)
10 responses2 = puf2.eval(challenges)
11 #Compute average Hamming distance between responses of the two PUFs .
12 uniqueness = np.mean(responses1 != responses2)
13 print("Average inter-chip Hamming distance:", uniqueness)
```

شکل 11- کد برای ارزیابی Uniqueness پاف Arbiter

همان طور که در کد هم مشاهده میکنید ابتدا دو Arbiter Puf با 64 استیج ایجاد کردیم و 1000 چلنج به شکل رندوم تولید کردیم ، در ادامه اومدیم و خروجی پاف ها یا همان ریسپانس ها رو با هم مقایسه کردیم.

طبیعتا همان طور که پیش تر نیز اشاره شد انتظار داریم که خروجی دو پاف با هم متفاوت باشد.

Uniqueness

Uniqueness is a measure for how differently PUF instances of the same class behave. As security is build on the individual behavior, a high uniqueness is a minimum security requirement for any PUF. The concept of uniqueness is also often called *inter-distance*. pypuf can approximate the uniqueness based on given instances of `Simulation` or based on response data, see below.

Uniqueness is estimated on a per-challenge basis, as low uniqueness on a small number of challenges can be problematic. To obtain a general uniqueness measure for each response bit, average results along the first axis.



```
Interactive - evaluate_reliability.py | Interactive - evaluate_uniqueness.py X ...
Interrupt | X Clear All | Restart | Jupyter Variables | qiQOURSE (Python 3.8.20)

Connected to qiQOURSE (Python 3.8.20)

✓ import numpy as np ...
... Average inter-chip Hamming distance: 0.492
```

شکل 12- خروجی ارزیابی Uniqueness برای Arbiter Puf

طبق خروجی گرفته شده این مقدار برای دو نمونه از Arbiter Puf عددی در حدود 50 درصد است که نشان میدهد که در نیمی از موارد دو پاف برای چلنج یکسان خروجی یکسان میدهد که از عدد ایده آل ما کمی فاصله دارد.

ارزیابی Uniformity برای Arbiter Puf:

Bias

The response bias is one of the fundamental metrics for the response behavior of PUFs. Large response bias enable trivial modeling attacks on the PUF as the attacker can just guess the more likely of the two responses.

PUF bias is in the literature also known as *balance* or *uniformity*.

Note that per bit encoding in pypuf, a bias of zero means a perfectly unbiased behavior, whereas bias of -1 and 1 means that the PUF under test always returns -1 and 1, respectively. To convert the pypuf bias value b into the more traditional 0-1-scale, use $1/2 - b/2$.

طبق توضیحات سایت <https://pypuf.readthedocs.io/en/latest/metrics/bias.html> این معیار بیان میکند که پاف ما به ازای چلنج های مختلف با چه مقیاسی جواب -1 یا 1 برمیگردونه و آیا در تولید جواب ریسپانس برای چلنج ها بایاس داره یا خیر.

که خب مشخصه که هر چی این مقدار به صفر نزدیک تر باشه ایده آل تره.

در صفحه بعد کد مربوط به این بخش به همراه توضیحات تکمیلی قرار داده شده است.

```
evaluate_reliability.py  evaluate_uniqueness.py  evaluate_uniformity.py X  evaluat
D: > Hardware security > CA&HWs > CA2 > evaluate_uniformity.py
1  from pypuf.simulation import ArbiterPUF
2  from pypuf.metrics import bias
3
4  bias(ArbiterPUF(n=64, seed=42), seed=1)
5
6
```

شکل 13- ایجاد فایل و کد برای ارزیابی Uniformity برای Arbiter Puf

کد بالا بر اساس تنظیمات و توضیحات در سایت pypuf نوشته شده است در زیر می‌توانید مثال ذکر شده در سایت رو هم مشاهده کنید.

Arbiter PUF simulations in pypuf per additive delay model almost unbiased:

```
>>> from pypuf.simulation import ArbiterPUF
>>> from pypuf.metrics import bias
>>> bias(ArbiterPUF(n=128, seed=42), seed=1)
-0.004
```

شکل 14- مثال سایت pypuf برای بایاس Arbiter puf

در زیر نیز خروجی این کد که در محیط VS code گرفته شده است قرار گرفته است:

```
Interrupt | X Clear All | Restart | Jupyter Variables | qiQOURSE (Python 3.8.20)
Connected to qiQOURSE (Python 3.8.20)
✓ from pypuf.simulation import ArbiterPUF ...
... -0.016
```

شکل 15- خروجی بایاس برای Arbiter Puf

خب همان طور که مشاهده میکنید این مقدار یک بایاس منفی بسیار کم را برای Arbiter Puf نشان میدهد که خب یعنی پاف ما به اندازه خیلی کمی تمایل به تولید خروجی ریسپانس -1 دارد، که البته این مقدار بسیار ناچیز و نزدیک صفر است.
در نتیجه پاف ما عملکرد خیلی خوبی از این نظر دارد.

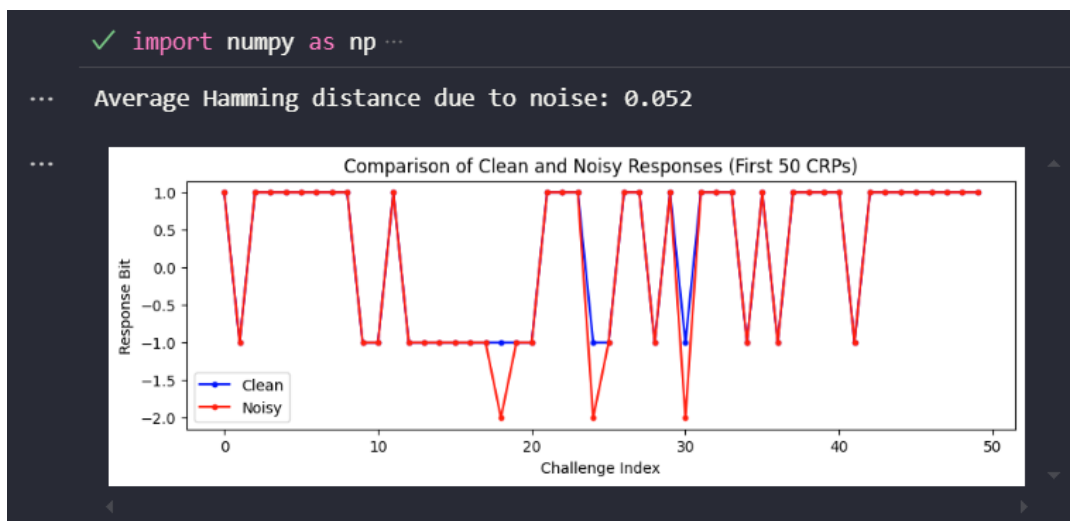
ارزیابی نویز برای Arbiter Puf

در این بخش به تحلیل تاثیر نویز در خروجی های Arbiter Puf میپردازیم. برای این منظور ابتدا یک فایل به نام simulated_niose.py ایجاد کردیم.

```
evaluate_reliability.py  evaluate_uniqueness.py X  evaluate_uniformity.py  simulated_niose.py X  ▾  □  ...
D: > Hardware security > CA&HWs > CA2 > simulated_niose.py > ...
1  import numpy as np
2  from pypuf.simulation import ArbiterPUF
3  import matplotlib.pyplot as plt
4  n_stages = 64
5  num_crps = 1000
6  puf = ArbiterPUF(n=n_stages)
7  challenges = np.random.randint(0, 2, size=(num_crps, n_stages))
8  responses_clean = puf.eval(challenges)
9  #Introduce noise: flip bits with a given probability .
10 noise_level = 0.05 # 5% noise
11 noise = np.random.rand(*responses_clean.shape) < noise_level
12 responses_noisy = np.bitwise_xor(responses_clean, noise.astype(int))
13 #Calculate average Hamming distance between clean and noisy responses .
14 noise_effect = np.mean(responses_clean != responses_noisy)
15 print("Average Hamming distance due to noise:", noise_effect)
16 #Plot a small segment of clean vs noisy responses .
17 plt.figure(figsize=(10, 3))
18 plt.plot(responses_clean[:50], 'b.-', label="Clean")
19 plt.plot(responses_noisy[:50], 'r.-', label="Noisy")
20 plt.legend ()
21 plt.title("Comparison of Clean and Noisy Responses (First 50 CRPs)")
22 plt.xlabel("Challenge Index")
23 plt.ylabel("Response Bit")
24 plt.show()
```

شکل 16- کد برای ایجاد نویز برای Arbiter Puf

همان طور که مشاهده میکنید در کد بالا در ابتدا 1000 چلنج به همراه ریسپانس ایجاد شده است و سپس با احتمال 5 درصد نویز روی خروجی ریسپانس افتاده است که در واقع بیت های ریسپانس رو فلیپ کرده است. در آخر نیز برای 50 زوج ابتدایی challenge-response بررسی شده است که آیا خروجی صحیح است یا نویز دار است.



شکل 17- خروجی نویز برای Arbiter Puf

خب همان طور که در نتایج مشخص است بیشتر خروجی ها نویز دار بوده است و تنها کسر کمی از خروجی ها سالم و بدون نویز بوده است که این نشان میدهد نویزهای محیطی به شدت روی خروجی این پاف تاثیرگذار است پس بنابراین نیاز است جهت از روشهای تصحیح خطا استفاده کنیم. در ضمن این نویز های محیطی در شرایط واقعی هم وجود دارن و روی عملکرد پاف تاثیر میذارن.

بررسی حمله ماشین لرنینگ به Arbiter Puf

در قسمت می‌خواهیم به بررسی و مقاومت Arbiter Puf در برابر حملات ماشین لرنینگ بپردازیم. جهت بررسی این مورد یک فایل با نام ml_attack.py ساختیم و کد مربوط به این بخش را در آن قرار دادیم:

```
1 import numpy as np
2 from sklearn.linear_model import LogisticRegression
3 from sklearn.model_selection import train_test_split
4 from sklearn.metrics import accuracy_score
5 #Load the previously saved CRPs
6 challenges = np.loadtxt("challenges.csv", delimiter=",", dtype=int)
7 responses = np.loadtxt("responses.csv", delimiter=",", dtype=int)
8 #Split data: 70% training, 30% testing .
9 X_train, X_test, y_train, y_test = train_test_split(challenges, responses, test_size=0.3,
10 random_state=42)
11 #Train a logistic regression model .
12 model = LogisticRegression(max_iter=1000)
13 model.fit(X_train, y_train)
14 #Predict on test set and evaluate accuracy .
15 predictions = model.predict(X_test)
16 accuracy = accuracy_score(y_test, predictions)
17 print("Machine Learning Attack Accuracy:", accuracy)
```

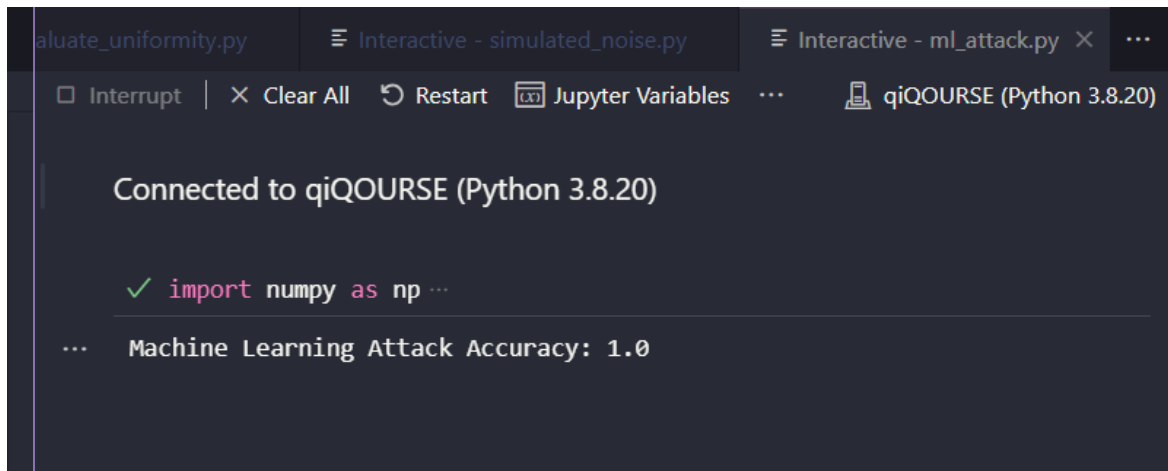
شکل 18- کد برای ML attack به Arbiter Puf

خب همان طور که در شکل بالا مشاهده می‌کنید ابتدا اومدیم و از فایل‌های چلنج و ریسپانس مقادیر رو لود کردیم که به عنوان داده برای train یک مدل logistic regression به کار می‌رود.

حال داده ها رو به دو بخش train و test با درصد 70 به 30 تقسیم کردیم که برای ترین کردن مدل لاجستیک رگرسیون از 70 درصد داده ها و با 1000 iteration استفاده شده است و 30 درصد داده ها هم برای تست این مدل است که ببینیم با چه کیفیتی این مدل خروجی ها را پیش بینی میکند.

این روش به Modeling Attack هم مربوط است که میاد و نحوه کار Puf رو مدل میکند.

حال در صفحه بعد خروجی این بخش قرار داده شده است.



```
Interactive - simulated_noise.py | Interactive - ml_attack.py | ...
Interrupt | Clear All | Restart | Jupyter Variables | qiQOURSE (Python 3.8.20)

Connected to qiQOURSE (Python 3.8.20)

import numpy as np ...

Machine Learning Attack Accuracy: 1.0
```

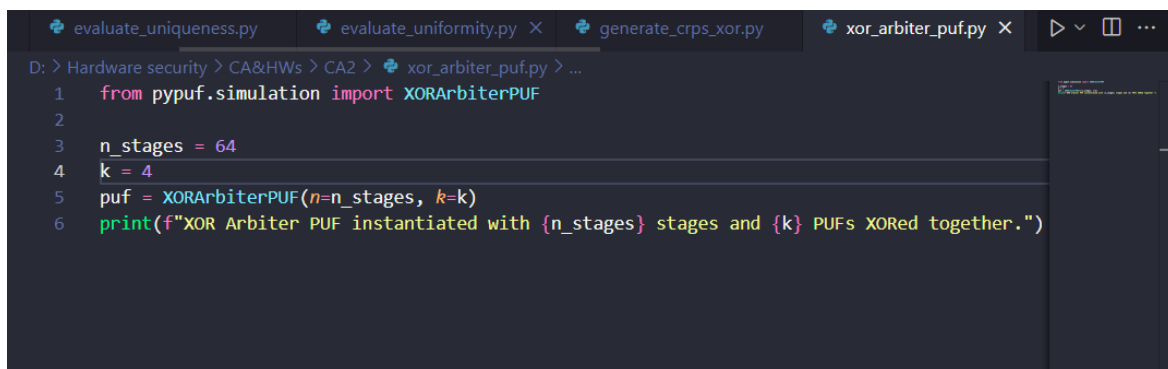
شکل 19- خروجی ML attack برای Arbiter Puf

خب همان طور که از خروجی هم مشخص است این پاف در برابر حمله ماشین لرنینگ به شدت ضعیف است و مدل ماشین لرنینگ logistic regression ما توانست دقت 100 درصد رو در پیش بینی جواب ها (response) داشته باشه.

بخش دوم: XOR Arbiter PUF

تولید XOR Arbiter Puf:

در ابتدا مثل روال قبل یک فایل با نام xor_arbiter_puf.py ایجاد میکنیم و یک نمونه از آن با 64 استیج و 4 XOR میسازیم.



```
evaluate_uniqueness.py | evaluate_uniformity.py | generate_crps_xor.py | xor_arbiter_puf.py | ...
D: > Hardware security > CA&HWs > CA2 > xor_arbiter_puf.py > ...
1 from pypuf.simulation import XORArbiterPUF
2
3 n_stages = 64
4 k = 4
5 puf = XORArbiterPUF(n=n_stages, k=k)
6 print(f"XOR Arbiter PUF instantiated with {n_stages} stages and {k} PUFs XORed together.")
```

شکل 20- ایجاد XOR Arbiter puf

در صفحه بعد میتونید خروجی مربوط به این فایل رو مشاهده کنید.


```

- simulated_noise.py  Interactive - ml_attack.py  Interactive - xor_arbiter_puf.py  ...
Interrupt | Clear All | Restart | Jupyter Variables | ... | qiQOURSE (Python 3.8.20)

Connected to qiQOURSE (Python 3.8.20)

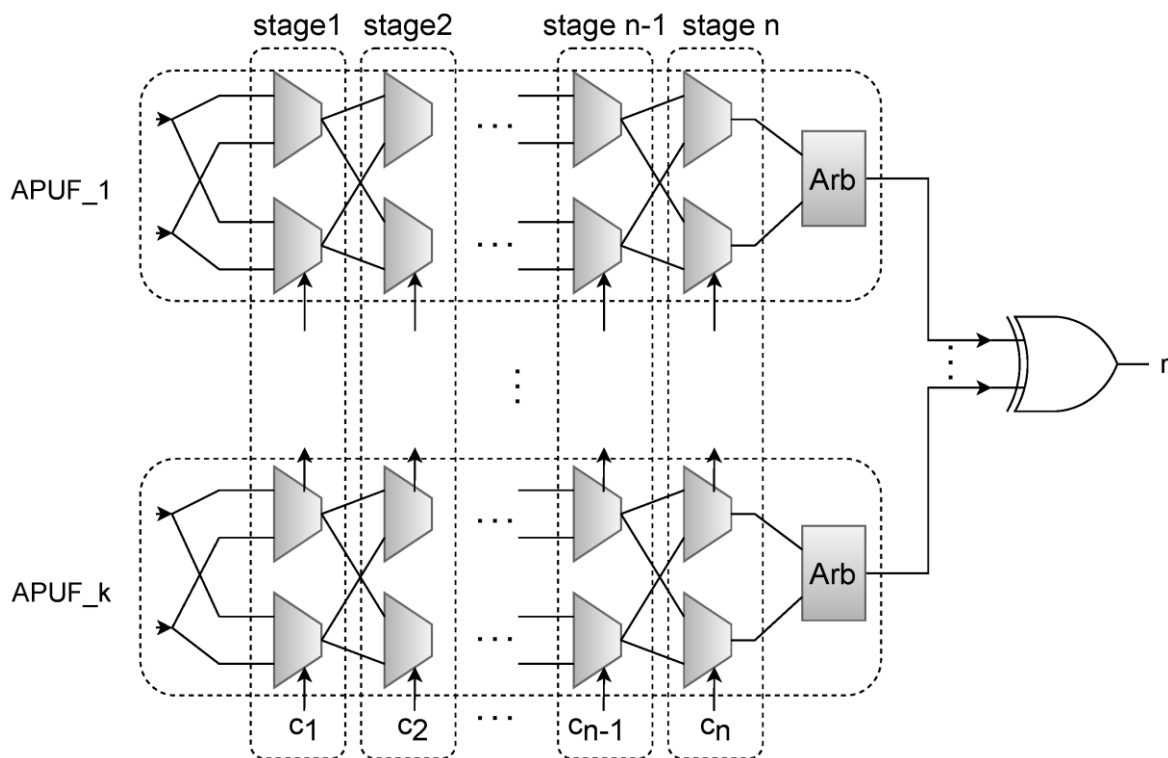
✓ from pypuf.simulation import XORArbiterPUF ...

... XOR Arbiter PUF instantiated with 64 stages and 4 PUFs XORed together.

```

شکل 21- خروجی فایل xor_arbiter_puf.py

در ادامه شکلی از XOR Arbiter PUF قرار داده شده است با کارکرد آن راحت تر آشنا شویم:



شکل 22- شکل XOR Arbiter PUF

همان طور که در شکل بالا نیز مشاهده می‌کنید نحوه کار مثل قبل است با اینکه در اینجا در انتها خروجی دو پاف با هم xor میشوند.

در مقایسه با Arbiter PUF معمولی این رویکرد معمولاً عملکرد بهتری دارد که در ادامه نیز مورد بررسی قرار خواهد گرفت.

تولید زوج challenge-response برای XOR Arbiter Puf

در این مرحله جهت تولید 10000 زوج چلنج ریسپانس ابتدا فایل generate_crps_xor.py را ایجاد کرده و پس از اجرا خروجی چلنج ریسپانس ها در فایل csv ممربوط به خود ریخته میشوند.

```
py evaluate_uniformity.py generate_crps_xor.py x xor_arbiter_puf.py simulated_noise.py
> Hardware security > CA&HWs > CA2 > generate_crps_xor.py > ...
1 import numpy as np
2 from pypuf.simulation import XORArbiterPUF
3
4 n_stages = 64
5 num_xors = 4 # های ترکیبی XOR تعداد
6 num_crps = 10000
7
8 # ساخت PUF
9 puf = XORArbiterPUF(n=n_stages, k=num_xors)
10
11 # تولید چالشها و پاسخها
12 challenges = np.random.randint(0, 2, size=(num_crps, n_stages))
13 responses = puf.eval(challenges)
14
15 # ذخیره در فایل
16 np.savetxt("xor_challenges.csv", challenges, fmt="%d", delimiter=",")
17 np.savetxt("xor_responses.csv", responses, fmt="%d", delimiter=",")
18 print("Generated", num_crps, "XOR CRPs.")
19
```

شکل 23- فایل generate-crps-xor

در ادامه خروجی این فایل قرار داده شده است:

```
Interrupt | X Clear All Restart Jupyter Variables ... qiQOURSE (Python 3.8.20)

Connected to qiQOURSE (Python 3.8.20)

✓ import numpy as np ...

... Generated 10000 XOR CRPs.
```

شکل 24- ایجاد 10000 زوج challenge-response

در ضمن لازم به ذکر است که به خاطر اینکه تابع bias از ورژن پایتون 3.8 به بعد در دسترس نیست لذل تمامی قایلها با پایتون 3.8 اجرا شده اند و مثلاً در خروجی ریسپانس ها به جای -1 و 0 مقادیر 1 و 1 موجود است.

در ادامه تست های مختلف جهت ارزیابی پاف قرار داده شده است.

ارزیابی Reliability برای XOR Arbiter Puf

در این مرحله برای ارزیابی intra-chip و reliability ابتدا یک فایل با نام `evaluate_reliability_xor` ایجاد می‌کنیم که در تصویر زیر می‌توانید کد مربوط به اون رو مشاهده کنید:

```
ess.py | evaluate_uniformity.py | generate_crps_xor.py | evaluate_reliability_xor.py | xor_arbiter_puf.py
D: > Hardware security > CA&HWs > CA2 > evaluate_reliability_xor.py > ...
1 import numpy as np
2 from pypuf.simulation import XORArbiterPUF
3
4 n_stages = 64
5 num_xors = 4
6 num_crps = 1000
7
8 puf = XORArbiterPUF(n=n_stages, k=num_xors)
9
10 challenges = np.random.randint(0, 2, size=(num_crps, n_stages))
11
12 responses1 = puf.eval(challenges)
13 responses2 = puf.eval(challenges)
14
15 reliability = np.mean(responses1 != responses2)
16
17 print("XOR PUF - Average intra-chip Hamming distance (Reliability):", reliability)
```

شکل 25- فایل `evaluate_reliability_xor.py`

خب در ادامه نیز کد مربوط به این کد قرار داده شده است:

```
Interrupt | Clear All | Restart | Jupyter Variables | qiQOURSE (Python 3.8.20)

Connected to qiQOURSE (Python 3.8.20)

✓ import numpy as np ...

... XOR PUF - Average intra-chip Hamming distance (Reliability): 0.0
```

شکل 26- خروجی کد ارزیابی reliability برای XOR Arbiter PUF

همان طور که در خروجی کد نیز مشخص است این پاف مثل `arbiter puf` معمولی که در بخش قبل مورد بررسی قرار داده شد عملکرد خیلی خوبی دارد و مقدار ایده آل 0 رو تولید کرده که نشان دهنده این است که به ازای دو چلنج یکسان در شرایط یکسان این پاف خروجی یکسانی تولید میکند که خب ما هم همین رو می‌خوایم که مقدار `intra-chip hamming distance` کم باشه.

در صفحه بعد به بررسی `inter-chip` برای این پاف می‌پردازیم

ارزیابی Uniqueness برای XOR Arbiter Puf

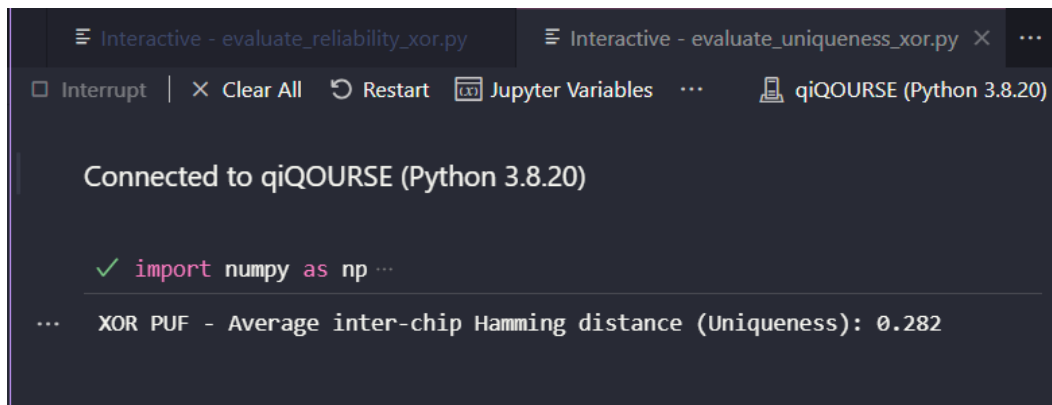
در این بخش نیز ابتدا یک فایل با نام `evaluate_uniqueness_xor.py` میسازیم که در زیر نیز کد آن قرار داده شده است:



```
1 import numpy as np
2 from pypuf.simulation import XORArbiterPUF
3
4 n_stages = 64
5 num_xors = 4
6 num_crps = 1000
7
8 puf1 = XORArbiterPUF(n=n_stages, k=num_xors, seed=1)
9 puf2 = XORArbiterPUF(n=n_stages, k=num_xors, seed=2)
10
11 challenges = np.random.randint(0, 2, size=(num_crps, n_stages))
12
13 responses1 = puf1.eval(challenges)
14 responses2 = puf2.eval(challenges)
15
16 uniqueness = np.mean(responses1 != responses2)
17
18 print("XOR PUF - Average inter-chip Hamming distance (Uniqueness):", uniqueness)
19
```

شکل 27- کد فایل `evaluate_uniqueness_xor.py`

در ادامه خروجی مربوط به این فایل هم قرار داده شده است:



```
Interactive - evaluate_reliability_xor.py | Interactive - evaluate_uniqueness_xor.py X ...
Interrupt | X Clear All | Restart | Jupyter Variables | qiQOURSE (Python 3.8.20)

Connected to qiQOURSE (Python 3.8.20)

✓ import numpy as np ...

... XOR PUF - Average inter-chip Hamming distance (Uniqueness): 0.282
```

شکل 28- خروجی فایل `evaluate_uniqueness` برای XOR Arbiter PUF

همان طور که در خروجی هم مشاهده میکنید این مقدار نسبت به پاف قبلی کمتر است و اصلا مطلوب نیست ما دوست داشتیم که مقدار `inter-chip` برای چیپ ها یا تفاوت بین پاف ها از یک نوع با ورودی چلنج یکسان زیاد باشد پس عدد ایده آل در اینجا مقدار 1.0 میباشد.

حال در بخش بعدی به بررسی بایاس میپردازیم:

ارزیابی Uniformity برای XOR Arbiter Puf

برای این بخش نیز از مثال سایت ذکر شده در صورت سوال استفاده کردیم:

On the other hand, 2-XOR Arbiter PUFs can have relatively large bias [WP20].

```
>>> from pypuf.simulation import XORArbiterPUF
>>> bias(XORArbiterPUF(n=64, k=2, seed=2), seed=2)
-0.086
```

حال یک فایل با نام evaluate_uniformity_xor.py میسازیم که کد آن را میتوانید در زیر مشاهده کنید:



```
D: > Hardware security > CA&HWs > CA2 > evaluate_uniformity_xor.py > ...
1  import numpy as np
2  from pypuf.simulation import XORArbiterPUF
3  from pypuf.metrics import bias
4
5  # تنظیم پارامترها
6  n_stages = 64
7  n_xors = 4
8  seed = 2
9
10 # ساخت PUF
11 puf = XORArbiterPUF(n=n_stages, k=n_xors, seed=seed)
12
13 # محاسبه bias
14 bias_value = bias(puf, seed=seed) # اعمال بر روی شی
15 uniformity = 1 - abs(bias_value)
16
17 print("XOR PUF - Bias:", bias_value)
18 print("XOR PUF - Uniformity:", uniformity)
```

شکل 29- کد فایل evaluate_uniformity_xor.py

در زیر نیز میتونید خروجی مربوطه رو مشاهده کنید:



```
Interactive - evaluate_uniqueness.py | Interactive - evaluate_uniformity_xor.py X | ...
Interrupt | X Clear All | Restart | Jupyter Variables | ... | qiQOURSE (Python 3.8.20)

Connected to qiQOURSE (Python 3.8.20)

✓ import numpy as np ...

... XOR PUF - Bias: 0.004
XOR PUF - Uniformity: 0.996
```

شکل 30- خروجی بایاس برای xor_arbiter puf

همان طور که مشاهده می‌کنید در مثالی که در سایت pypuf موجود است مقدار بایاس برای این پاف از پاف قبلی بسیار بیشتر است ولی با افزایش تعداد XOR در پاف تونستیم مقدار بایاس را کاهش دهیم و حتی از پاف Arbiter معمولی بهتر شد.

ارزیابی نویز برای XOR Arbiter Puf

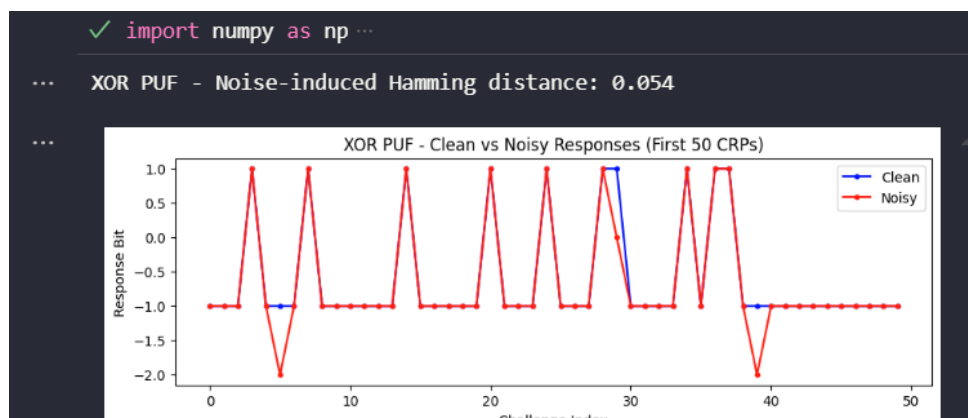
برای این مورد هم مثل قبل عمل میکنیم و با اعمال نویز روی خروجی ریسپانس پاف به مشاهده نتایج میپردازیم:

```

generate_crps_xor.py  evaluate_reliability_xor.py  evaluate_uniformity_xor.py  simulate_noise_xor.py  evalu...
D: > Hardware security > CA&HWs > CA2 > simulate_noise_xor.py > ...
1  import numpy as np
2  from pypuf.simulation import XORArbiterPUF
3  import matplotlib.pyplot as plt
4
5  n_stages = 64
6  num_xors = 4
7  num_crps = 1000
8  noise_level = 0.05 # 5% نویز
9
10 puf = XORArbiterPUF(n=n_stages, k=num_xors)
11
12 challenges = np.random.randint(0, 2, size=(num_crps, n_stages))
13 responses_clean = puf.eval(challenges)
14
15 # افزودن نویز
16 noise = np.random.rand(*responses_clean.shape) < noise_level
17 responses_noisy = np.bitwise_xor(responses_clean, noise.astype(int))
18
19 noise_effect = np.mean(responses_clean != responses_noisy)
20
21 print("XOR PUF - Noise-induced Hamming distance:", noise_effect)
22
23 plt.figure(figsize=(10, 3))
24 plt.plot(responses_clean[:50], 'b.-', Label="Clean")
25 plt.plot(responses_noisy[:50], 'r.-', Label="Noisy")
26 plt.legend()
27 plt.title("XOR PUF - Clean vs Noisy Responses (First 50 CRPs)")
28 plt.xlabel("Challenge Index")
29 plt.ylabel("Response Bit")
30 plt.show()

```

در نهایت خروجی این کد در زیر قرار داده شده است:

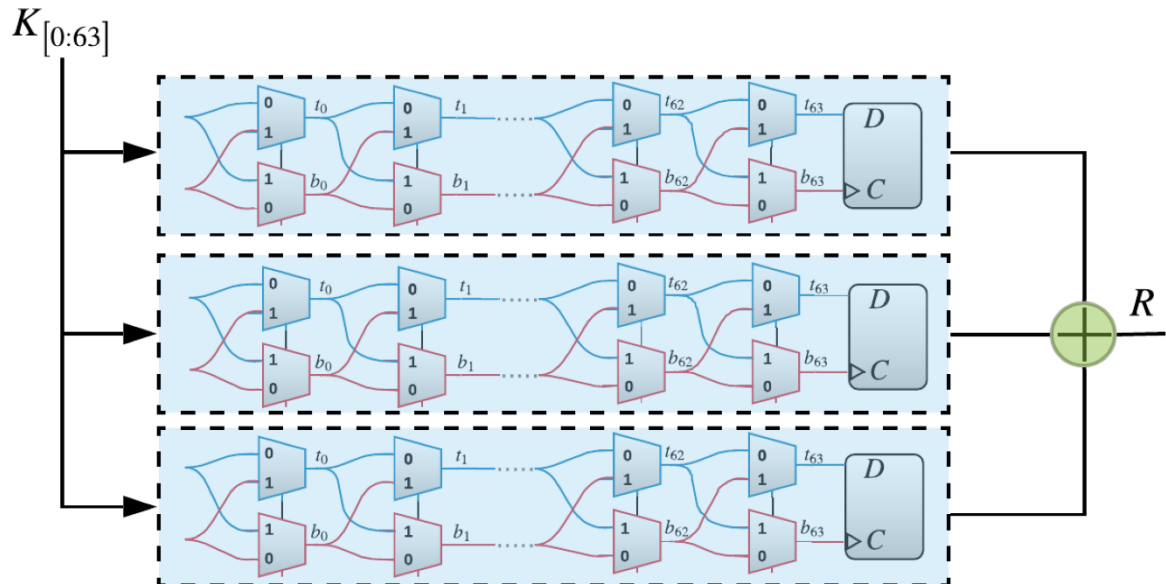


شکل 31- خروجی نویز برای XOR Arbiter PUF

همان طور که در تصویر مشاهده میکنید در اینجا کمی وضعیت بهتر است اما در کل اینجا هم بسیار حساس به نویز است و مشکل پاف قبلی را دارد.

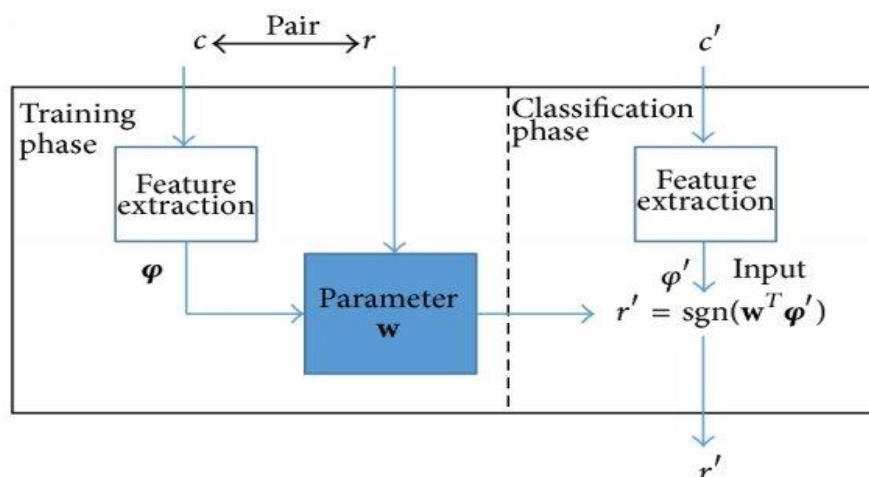
بررسی حمله ماشین لرنینگ به XOR Arbiter Puf:

در این قسمت میخواهیم مشابه قبل مقاومت پاف رو در برابر حمله ML مورد بررسی قرار دهیم.



شکل 32- تصویری از xor arbiter puf

در زیر نیز تصویری از نحوه حمله به این پاف از طریق ماشین لرنینگ قرار داده شده است:



در ادامه میتونید کد مربوط به این قسمت به همراه خروجی آن را مشاهده کنید:

```

generate_crps_xor.py  evaluate_reliability_xor.py  ml_attack_xor.py  evaluate_uniformity_xor.py
D: > Hardware security > CA&HWs > CA2 > ml_attack_xor.py > ...
1 import numpy as np
2 from sklearn.linear_model import LogisticRegression
3 from sklearn.model_selection import train_test_split
4 from sklearn.metrics import accuracy_score
5
6 # تولید شده اند XORArbiterPUF که قبلاً توسط های ذخیره شده CRP بارگذاری
7 challenges = np.loadtxt("xor_challenges.csv", delimiter=",", dtype=int)
8 responses = np.loadtxt("xor_responses.csv", delimiter=",", dtype=int)
9
10 # تقسیم‌بندی داده‌ها به آموزش و آزمون (70% آموزش، 30% آزمون)
11 X_train, X_test, y_train, y_test = train_test_split(
12     challenges, responses, test_size=0.3, random_state=42
13 )
14
15 # ساخت و آموزش مدل رگرسیون لجستیک
16 model = LogisticRegression(max_iter=1000)
17 model.fit(X_train, y_train)
18
19 # پیش‌بینی پاسخ‌های آزمون و محاسبه دقت
20 predictions = model.predict(X_test)
21 accuracy = accuracy_score(y_test, predictions)
22
23 print("XOR Arbiter PUF - ML Attack Accuracy:", accuracy)
24

```

شکل 34- کد فایل ML attack برای xor arbiter puf

در نهایت میتونید کد مربوط به خروجی این فایل رو نیز مشاهده کنید:

```

ormity_xor.py  Interactive - simulate_noise_xor.py  Interactive - ml_attack_xor.py  ...
Interrupt | X Clear All Restart Jupyter Variables ... qiQOURSE (Python 3.8.20)

Connected to qiQOURSE (Python 3.8.20)

✓ import numpy as np ...

... XOR Arbiter PUF - ML Attack Accuracy: 0.9913333333333333

```

شکل 33- خروجی مربوط به ml attack برای XOR Arbiter PUF

همان طور که در اینجا مشاهده میکنید دقت حمله ماشین لرنینگ نسبت به قبل کاهش یافت و به مقدار 99 درصد رسیده است که از پاف قبلی بهتر است ولی همچنان بسیار در برابر این نوع حمله ضعیف است.

در ادامه در قالب یک بخش به مقایسه تک تک پارامترهای ارزیابی برای دو پاف میپردازیم.

بخش سوم: مقایسه دو پاف با یکدیگر

در زیر در قالب یک جدول به مقایسه دو پاف Arbiter PUF و XOR Arbiter PUF با یکدیگر میپردازیم:

جدول 1- مقایسه Arbiter PUF و XOR Arbiter PUF

	Arbiter PUF with 64 stage	XOR Arbiter PUF (4 XOR)
Reliability	0.00	0.00
Uniqueness	0.492 (%49)	0.282 (%28)
Uniformity(bias)	-0.016 (~%0)	0.004 (~%0)
noise	0.052	0.054
ML attack	1.0 (%100)	0.9913 (%99)

همان طور که در نتایج مشخص است دو پاف عملکرد مشابهی از نظر reliability دارند و در این مورد به عدد ایده آل ما رسیده اند (ما دوست داریم به ازای یک چلنج در شرایط یکسان هر بار یک خروجی یکسان بگیریم). از حیث uniqueness هم عملکرد Arbiter PUF بسیار بهتر است یعنی به ازای دو پاف از این نوع برای یک چلنج در نیمی از موارد خروجی متفاوت میگیریم که مطلوب است اما همچنان از ایده آل به دور است (ما دوست داریم در تمامی موارد به ازای یک چلنج برای دو پاف خروجی متفاوت بگیریم).

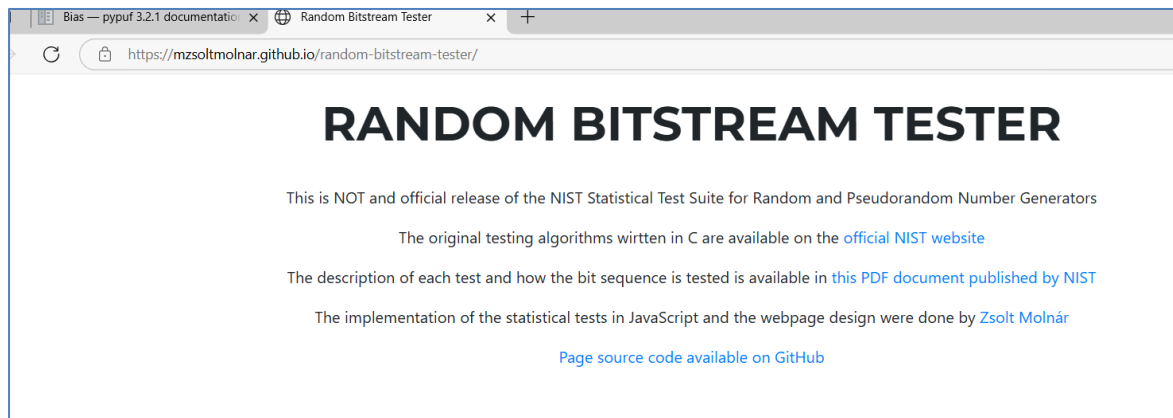
در کیس بایاس ما با 4 عدد XOR امتحان کردیم و بایاس بهتری گرفتیم در حالیکه اگر با 2 عدد XOR تست میکردیم خروجی بدتر میشد ولی در کل بایاس هر دو پاف خوب و نزدیک به صفر است. در مورد نویز نیز عملکرد دو پاف مثل هم است و نویز تاثیر زیادی در خروجی میگذارد. در مورد آخر که مربوط به حمله ماشین لرنینگ است نیز هر دو پاف بسیار آسیب پذیر هستند هر چند XOR Arbiter PUF به خاطر ساختار XOR ای که دارد تا حد کمی تونسته آمار رو بهبود دهد.

بخش چهارم: تحلیل تصادفی بودن Response ها و کاربرد PUF به

عنوان TRNG

یکی از کاربردهای مهم PUF ها استفاده از Response های تولید شده به عنوان اعداد تصادفی و در نتیجه استفاده از PUF به عنوان یک TRNG مستقل یا منبعی برای افزایش Randomness در سایر RNG ها میباشد. فرض کنید میخواهیم از یک String شامل تمام Response های جمع آوری شده در مرحله 2 و به همین ترتیب در مرحله 5 به عنوان دو کلید منحصر به فرد برای استفاده در

یک برنامه احراز هویت استفاده کنیم. از آنجایی که هر قدر Randomness مقادیر بیت‌های این کلید بیشتر باشد، احتمال شکستن آن توسط یک شخص مهاجم کمتر میشود، در این بخش میخواهیم میزان تصادفی بودن کلیدهای به دست آمده خود را بسنجیم



شکل 36- سایت RANDOM BITSTREAM TESTER

بعد از ورود به سایت Response های Arbiter PUF خود را به شکل یک string از بخش select input و به شکل manual وارد میکنیم.

شکل 35- نحوه وارد کردن رشته Response ها در سایت

آزمون فراوانی (Frequency Test): بررسی میکند که آیا تعداد بیت‌های 0 و 1 تقریباً برابر است یا خیر

• آزمون بلاک فراوانی (Block Frequency Test): بررسی توزیع بیت‌ها در بخش‌های کوچک داده

• آزمون سری (Runs Test): ارزیابی تعداد سری‌های پیوسته بیت‌های 0 یا 1

• آزمون پیچیدگی خطی (Linear Complexity Test): اندازه‌گیری دشواری تولید یک دنباله خطی

که خروجی داده‌ها را تولید کند

خروجی های تست گرفته شده برای Arbiter PUF رو میتونید در صفحه بعد مشاهده کنید.

نتایج تست برای Arbiter PUF جهت تولید TRNG از response های تولید شده:

Tests Start Test		
Test name	Result value (P-value)	Status
1. Frequency (Monobit) Test	0.4592999943295808	Passed
2. Frequency Test within a Block	0.8487476147689124	Passed
3. Runs Test	0.20567591987936096	Passed
4. Test for the Longest Run of Ones in a Block	0.1623889678304961	Passed
5. Binary Matrix Rank Test		Error
6. Non-overlapping Template Matching Test		Error
7. Overlapping Template Matching Test		Error

شکل 37- نتایج برای 7 آزمون اول برای Arbiter PUF

همان طور که مشاهده میکنید خروجی تست نشان میدهد که 4 مورد از 7 آزمون ابتدایی رو تونسته رشته ریسپانس برای Arbiter PUF ما پاس کند که توضیحات سه مورد ابتدایی در صورت سوال نیز ذکر شده است.

در مورد Frequency Test نشان میدهد که تقریباً تعداد صفر ها با یک ها در رشته ورودی ما یکسان است و از این نظر بایاس وجود ندارد.

در مورد Frequency Test within a Block هم به عدد قابل قبولی رسیدیم و تست پاس شده است اما طبیعتاً تعداد صفر و یک ها در هر بلاک برابر نیست.

در مورد Runs Test هم به عدد 20 درصد رسیدیم و نشان میدهد که MAX تعداد صفر ها یا یک های پشت سر هم برابر با 20 درصد است.

در مورد تست چهارم هم مثل قبل است با اینکه ماکسیمم تعداد 0 های پشت سر هم را بررسی کرده که برابر با 16 درصد است و عدد قابل قبولی محسوب میشود.

بخش دوم نتایج آزمایش ها در صفحه بعد قرار داده شده است.

8. Maurer's "Universal Statistical" Test		Error
9. Linear Complexity Test		Error
10. Serial Test		Error
11. Approximate Entropy Test	0.5852409697352372	Passed
12. Cumulative Sums (Cusum) Test	P-value Forward: 0.3351038285061634 P-value Reverse: 0.7967274222814122	Passed
13. Random Excursions Test		Error
14. Random Excursions Variant Test		Error

شکل 38- نتایج برای 7 آزمون دوم برای Arbiter PUF

در اینجا هم همان طور که مشاهده میکنید تنها دو تست از 7 تست پاس شدن که مربوط به Arbiter puf هستند.

به شکل کلی Entropy به بی نظمی سیستم گفته میشه و در اینجا منظور از اندازه رندوم بودن خروجی هاست که مشاهده میکنید این تست نیز پاس شده است.

در مورد Linear Complexity اما رشته ورودی ما نتونسته خوب عمل کنه و این تست FAIL شده است. این مورد به این می‌پردازد که دشواری تولید یک دنبایه خطی که خروجی ها را تولید کند چقدر است.

نتایج تست برای XOR Arbiter PUF جهت تولید TRNG از response های تولید شده:

خب برای پاف XOR Arbiter PUF هم میایم و خروجی ریسپانس های تولیدی رو به شکل رشته به شکل دستی در سایت وارد میکنیم و سپس گزینه Run Test رو میزنیم تا تست ها انجام شود.

در صفحه بعد میتونید خروجی تست ها رو نیز مشاهده کنید:

Bitstream input

Select the input type

☐ Radioactive decay RNG
☐ ADC noise RNG
☐ JavaScript pseudo RNG
☒ Manual bitstream input

Insert Input Stream

Input bitstream

Copy to Clipboard

```

10110000101101010111011110001011001010101001100010001010011101000101110001011011000
101111100100010000101110111100110101010101000101100111000001011000000101010001011000
1010010000011101001111010001000000101001011101010000110001110010010101000010000
010010100111000010001000110011110111110001001011101110110001111111101110011
1100110010110110011001011010100110011001100011110100111000001011001010000100101
11101011100011001111101100000100001100101111000110011011001111011010100011010
00100011011101101000011001000001011101010001011101000101001011000011011110001001
11111101111011100001010001011010110000111100100111101011100100001100111100001001
010001000111111100011110000111101010100001000101010111011010010101011111001110
11111101100100110010110001000001010000110010000011100100100000111001001000001100

```

Tests

Start Test

شکل 39- وارد کردن رشته ریسپانس های XOR Arbiter PUF در سایت

نکته مهم: برای اینکه در پایتون 3.8 خروجی ریسپانس ها به صورت -1 و 1 بود و اینجا نیاز بود تا به شکل 0 و 1 باشد یکبار دیگر کد های مربوطه در پایتون 3.12 ران گرفته شدند.

دلیل استفاده از پایتون 3.8 نبود لایبرری BIAS در ورژن های بالاتر بوده است و تمامی نتایجی که در مراحل قبل گرفته شده است با اجرا در Kernel Python 3.8 بوده است.

حال به بررسی نتایج میپردازیم:

1. Frequency (Monobit) Test	0.40090838652089944	Passed
2. Frequency Test within a Block	0.12399735287199726	Passed
3. Runs Test	0.03509984106845376	Passed
4. Test for the Longest Run of Ones in a Block	0.9218335955605758	Passed
5. Binary Matrix Rank Test		Error
6. Non-overlapping Template Matching Test		Error
7. Overlapping Template Matching Test		Error

شکل 40- خروجی 7 تست اول برای XOR Arbiter PUF

همان طور که در نتایج تست ها مشاهده میکنید نتایج مشابه قبل است مثلا از حیث Frequency Test تعداد صفر و یک ها و فراوانی آنها در رشته تقریبا یکسان است با اینکه در اینجا بایاس بیشتری رو مشاهده میکنیم ولی همچنان قابل قبول است.

در مورد Frequency in a block هم که به بررسی توزیع بیت ها در یک بلاک میپردازد نیز تونسته رشته ورودی ما آزمون رو پاس کند.

در بخش Runs Test اما عملکرد بهتری از قبل داشتیم و تعداد سریهای پشت سر هم 0 و 1 ها مقدار کمتری شد که مطلوب است و تونسته مثل خروجی پاف قبلی این تست رو نیز پاس کند.

در مورد Longest Runs of Ones in a block هم که به تعداد یک های پشت سر هم در یک بلاک اشاره دارد هم تونستیم این تست رو پاس کنیم مشابه قبل.

8. Maurer's "Universal Statistical" Test		Error
9. Linear Complexity Test		Error
10. Serial Test		Error
11. Approximate Entropy Test	0.1063685252749999	Passed
12. Cumulative Sums (Cusum) Test	P-value Forward: 0.20200860301216772 P-value Reverse: 1	Passed
13. Random Excursions Test		Error
14. Random Excursions Variant Test		Error

شکل 41- خروجی 7 تست دوم برای XOR Arbiter PUF

در این بخش نیز مشابه قبل رشته ریسپانس ورودی ما تونسته 2 تست را پاس کند.

یکی از تست ها مربوط به آنتروپی یا همان رندوم بودن تولید خروجی هاست که پاس شده است با اینکه در اینجا مقدار کمتری نسبت به قبلی دارد.

و به مانند قبل تست Linear Complexity Test رو نتونستیم پاس کنیم.

در مجموع با توجه به تست های گرفته شده از هر دو خروجی میتوان به عنوان کلید در کریپتوگرافی استفاده کرد. همچنین برای تولید TRNG هم مناسب است.

((پایان))